



Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent` .
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos` . Call your `astar_search` method and store the returned list of actions in `self.plan` .
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()` . Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS prioritizes the node with the lowest accumulated path cost $g(n)$. It uses only the cost already spent to reach the node, so it has no estimate of how far the node is from the goal. A* prioritizes the node with the lowest $f(n) = g(n) + h(n)$, where $h(n)$ estimates the remaining cost to the goal. Therefore, UCS is guided only by the cost so far, while A* combines the cost so far with an estimate of the cost still required.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

In this grid the agent can move only up, down, left, or right, and each move has a cost of 1. For two positions, Manhattan Distance is $|x1 - x2| + |y1 - y2|$. This is a lower bound on the true shortest path because the agent must make at least that many horizontal and vertical moves to close the coordinate difference. Walls can only force extra movement, so the real path cost can be equal to or greater than the Manhattan value. Therefore the heuristic never overestimates and is admissible. If the heuristic were not admissible, it could overestimate the remaining cost and A* would lose its guarantee of returning an optimal path. The algorithm could still find a valid path, but it might return a more expensive path.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, not when a diagonal move has the same unit cost as a horizontal or vertical move. For example, from (0, 0) to (1, 1), Manhattan Distance is 2, but an 8-way agent can reach the goal in one diagonal move. The heuristic would therefore overestimate the true cost and would not be admissible. For the usual 8-way unit-cost model, the heuristic should be changed to Chebyshev Distance, $\max(|dx|, |dy|)$, because one diagonal move can reduce both coordinate differences at once. If the implementation instead assigns a diagonal cost of $\sqrt{2}$, an appropriate admissible metric is the octile distance (and Euclidean distance is also admissible).

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A stronger heuristic is to estimate the minimum cost needed to connect the agent to all remaining food items. One practical approach is to build a Minimum Spanning Tree (MST) over the current agent position and all remaining food positions. Use Manhattan distances between the points as the edge costs (or precomputed shortest-path distances when more accuracy is worth the extra computation). The MST cost is a lower bound because any route that visits every food item must connect all of those locations. This gives A* information about the total remaining collection task instead of only the nearest food. Computing the exact optimal order of visiting every food item is much more expensive because it becomes a multi-goal routing problem similar to the travelling-salesperson problem; the MST provides a useful and much cheaper admissible lower bound.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

